

Projet

Gestion d'une bibliothèque en ligne (MongoDB)

Participants :

Amina ZOUANE, Yelizaveta SAMUILAVA, Sarah LAKROUZ, Ilayda ABDOUL

Table des matières

1 Enoncé du projet	3
2 Evaluation du Workflow de l'application	3
2.1 Utilisateurs et informations	3
2.1.1 Identification des entités - livres	3
2.1.2 Identification des entités - auteurs	4
2.1.3 Identification des entités - catégories	4
2.1.4 Identification des entités - utilisateurs	4
2.1.5 Identification des entités - emprunts	5
2.2 Quantification	5
2.3 Journaux d'activité et requêtes fréquentes	6
2.4 Identification des opérations Read/Write et Définition de la charge de travail	6
2.5 Modélisation de MongoDB	8
3 Définition des relations entre entités et choix embedded/linked	9
3.1 Grille de décision : Embedded ou Référence ?	9
4 Explication des requêtes sans l'agrégation	13
5 Explication des requêtes avec l'agrégation	16
6 Tableau des charges de travail	22
7 Travail avec Postman	23
8 Conclusion	24

1 Enoncé du projet

Dans ce projet, nous présentons une bibliothèque en ligne qui est gérée grâce à une base de données NoSQL de type document, administrée par MongoDB.

Sur notre site, les utilisateurs peuvent consulter les livres disponibles, les emprunter ou effectuer des recherches par auteur ou par catégorie selon leurs intérêts.

De plus, toutes les relations sont modélisées et les choix entre documents imbriqués (embedded) et références sont justifiés.

Dépôt GitHub :

[Yelizaveta-Samuilava/Bibliotheque_Projet_INF3](https://github.com/Yelizaveta-Samuilava/Bibliotheque_Projet_INF3)

2 Evaluation du Workflow de l'application

2.1 Utilisateurs et informations

2.1.1 Identification des entités - livres

L'entité principale est le livre, qui peut être emprunté par les utilisateurs. Il existe plusieurs types de livres, tels que : le livre électronique (eLivre), le livre audio et le livre imprimé.

eLivre	Livre audio	Livre imprimé
SKU	SKU	SKU
Titre	Titre	Titre
Auteur	Auteur	Auteur
Éditeur	Éditeur	Éditeur
Langue	Langue	Langue
Résumé	Résumé	Résumé
Date de publication	Date de publication	Date de publication
Nombre de pages	Durée (en minutes)	Nombre de pages

2.1.2 Identification des entités - auteurs

L'entité Auteur représente les personnes qui ont écrit les livres disponibles dans la bibliothèque. Chaque auteur peut être référencé par plusieurs livres.

Auteur
auteur_id
nom
Année de naissance
Biographie

2.1.3 Identification des entités - catégories

Chaque livre peut appartenir à une catégorie spécifique. L'entité Catégorie a donc été créée pour représenter ces classifications. Elle permet d'organiser les livres par thème et de faciliter la recherche pour les utilisateurs.

Catégorie
category_id
nom
description

2.1.4 Identification des entités - utilisateurs

Pour utiliser l'application de la bibliothèque, il est nécessaire de se connecter. L'entité Utilisateur a donc été créée pour représenter chaque personne inscrite. Cette entité stocke notamment les informations personnelles ainsi que le mot de passe unique de connexion.

Utilisateur
utilisateur_id
nom
prenom
email
mot_de_passe
rôle

2.1.5 Identification des entités - emprunts

Pour gérer les interactions liées aux emprunts de livres, l'entité Emprunt est nécessaire. Elle permet de suivre quels utilisateurs ont emprunté quels livres, ainsi que les dates et le statut de chaque emprunt.

Emprunt
emprunt_id
utilisateur_id
livre_id
date_emprunt
date_retour
statut

Au final, nous avons 7 entités pour notre application:

- eLivre, Livre audio, Livre imprimé
- Auteur
- Catégorie
- Utilisateur
- Emprunt

2.2 Quantification

L'opération de quantification consiste à estimer approximativement la charge de travail de notre application, dans le but de préparer la prochaine génération du workflow.

Entité	Quantité estimée	Quantité réelle de la base des données
eLivre	50000	100
Livre audio	2000	100
Livre imprimé	100000	100
Auteur	56000	40
Catégorie	100	20
Utilisateur	10000	5
Emprunt	20000	6

2.3 Journaux d'activité et requêtes fréquentes

Les journaux d'activité permettent de suivre l'utilisation de l'application, tandis que les requêtes fréquentes correspondent aux opérations les plus utilisées sur la base de données.

Journaux d'activité (logs) :

- Connexion des utilisateurs
- Emprunts et retours de livres
- Consultation des livres
- Recherches par auteur ou catégorie

2.4 Identification des opérations Read/Write et Définition de la charge de travail

La charge de travail de l'application bibliothèque est déterminée par les opérations que les utilisateurs et le système effectuent le plus fréquemment. Ces opérations peuvent être de lecture (Read), lorsqu'elles consultent des informations, ou d'écriture (Write), lorsqu'elles modifient ou ajoutent des données.

Fonctionnalités classiques (sans agrégation):

1. Gestion des livres : Ajouter, modifier, supprimer ou consulter les informations d'un livre (titre, auteur, année, catégorie, stock).
2. Gestion des utilisateurs : Création de compte utilisateur : les étudiants et administrateurs.
3. Emprunts et retours de livres : Mettre à jour la disponibilité, les dates d'emprunt/retour et l'état (en retard, rendu, en cours).
4. Recherche avancée : Permettre de rechercher un livre par titre, auteur, catégorie, mots-clés ou disponibilité.
5. Gestion automatique des retards : Détecter les emprunts dépassant la durée maximale.

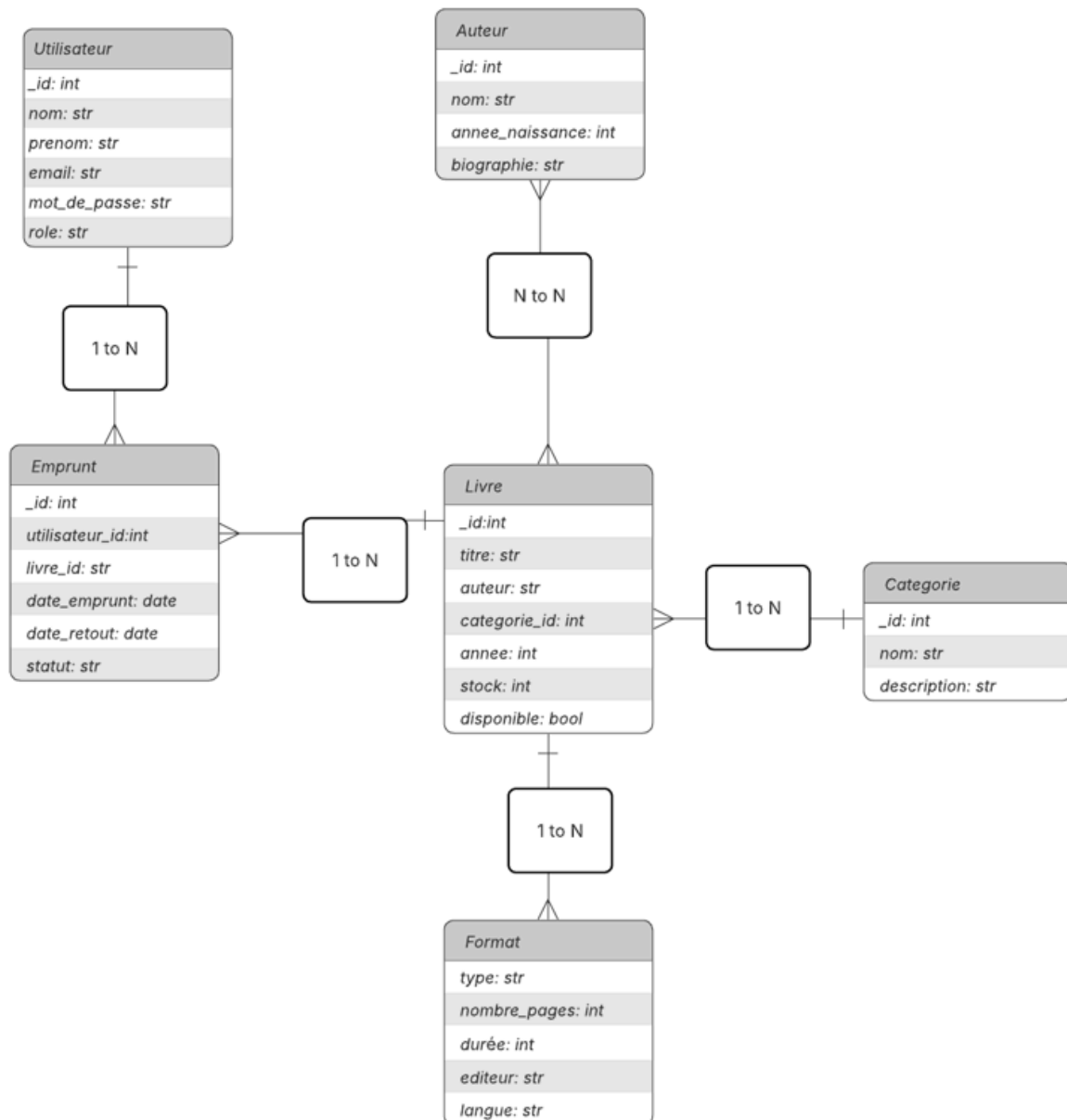
Fonctionnalités analytiques (requêtes avec agrégations)

1. Top des livres les plus empruntés : Identifier les livres les plus populaires.
2. Nombre d'emprunts par catégorie : Obtenir le total d'emprunts pour chaque catégorie de livre.
3. Durée moyenne d'emprunt par utilisateur : Calculer combien de temps en moyenne chaque utilisateur garde un livre.
4. Durée moyenne d'emprunt par livre : Calculer la durée moyenne de chaque livre emprunté.
5. Top des écrivains les plus empruntés : Identifier les auteurs dont les livres sont le plus empruntés.

Entités	Opérations	Informations Requises	Type	Fréquence	Priorité
Livres	Récupérer détails livre	Détails livre + disponibilité dans le catalogue	Lecture	1 000 fois par jour	Haut
Utilisateurs	Récupérer détails utilisateur	Détails utilisateur	Lecture	100 fois par jour	Moyen
Utilisateur	Création de compte	Détails utilisateur	Ecriture	10 fois par jour	Haut
Auteurs + Livres	Récupérer auteur et ses livres	Titres des livres + détails auteur	Lecture	800 fois par jour	Moyen
Livres	Ajouter/ mettre à jour un livre	Détails livre	Ecriture	50 fois par jour	Moyen
Catégorie + Livres	Récupérer détails livre par catégorie	Détails livre + catégorie	Lecture	900 fois par jour	Moyen
Emprunts	Récupérer les retards des emprunts	Détails emprunts	Lecture	900 fois par jour	Haut
Livres + emprunts	Afficher les livres le plus empruntés	Détails emprunts + livres	Lecture	100 fois par jour	Bas
Emprunts + utilisateur + livre	Emprunter un livre	Détails emprunts+livres + utilisateurs	Ecriture	1 000 fois par jour	Haut
Emprunts + Utilisateur	Afficher la durée moyenne d'emprunt par utilisateur	Détails emprunts + utilisateurs	Lecture	100 fois par jour	Bas
Emprunts + Livres	Afficher la durée moyenne d'emprunt par livre	Détails emprunts+livres	Lecture	100 fois par jour	Bas
Emprunts + Auteurs	Afficher le top des écrivains les plus empruntés	Détails emprunts+auteurs	Lecture	100 fois par jour	Bas

2.5 Modélisation de MongoDB

Le schéma de la base de données a été conçu pour représenter de façon claire les différents éléments de l'application de bibliothèque en ligne. On y retrouve les utilisateurs, les livres, les auteurs, les catégories, les emprunts et les formats. Les relations entre ces entités reflètent le fonctionnement réel du système : un utilisateur peut emprunter plusieurs livres, un livre appartient à une catégorie, peut avoir plusieurs formats et être associé à un ou plusieurs auteurs. Cette organisation permet de gérer efficacement les données tout en facilitant les recherches et les différentes fonctionnalités de l'application.



3 Définition des relations entre entités et choix embedded/linked

3.1 Grille de décision : Embedded ou Référence ?

Pour concevoir la base de données MongoDB de la bibliothèque, il est nécessaire de décider quelles relations entre entités seront imbriquées (Embedded) et lesquelles seront référencées (Linked/Referenced). Cette décision se base sur les critères suivants : fréquence des requêtes, cardinalité, évolutivité, mises à jour simultanées, individualité, duplication tolérable, archivage et simplicité.

1. Relation : Livre ↔ Auteur

Type : Référence

Justification :

- Un auteur peut écrire plusieurs livres (cardinalité élevée).
- Un auteur existe indépendamment d'un livre.
- Les auteurs sont souvent consultés séparément (par exemple : afficher tous les livres d'un auteur).
- Une mise à jour sur un auteur ne doit pas nécessiter la mise à jour de tous les livres de la bibliothèque

Listing 1 – Livre avec auteur (référence)

```
{
  "_id": 1,
  "titre": "Introduction à MongoDB",
  "auteur_id": 1,
  "categorie_id": 1,
  "annee": 2023,
  "stock": 5,
  "disponible": true
},
```

Listing 2 – Auteur stocké séparément

```
{
  "_id": 1,
  "nom": "Jean Dupont",
  "annee_naissance": 1975,
  "biographie": "Jean Dupont est un écrivain français contemporain connu pour ses romans et es",
},
```

2. Relation : Livre ↔ Catégorie

Type : Référence

Justification :

- Une catégorie peut contenir de nombreux livres.
- Une catégorie peut changer sans devoir modifier chaque livre (mise à jour indépendante).
- Évite la duplication des informations de catégorie.
- Les catégories sont une entité autonome.

Listing 3 – Livre avec catégorie (référence)

```
{
  "_id": 1,
  "titre": "Introduction à MongoDB",
  "auteur_id": 1,
  "categorie_id": 1,
  "annee": 2023,
  "stock": 5,
  "disponible": true
},
```

Listing 4 – Catégorie stocke séparément

```
{
  "_id": 1,
  "nom": "Jean Dupont",
  "annee_naissance": 1975,
  "biographie": "Jean Dupont est un écrivain français contemporain connu pour ses romans et es:
},
```

3. Relation : Utilisateur ↔ Emprunts

Type : Embedded

Justification :

- Les emprunts d'un utilisateur sont souvent consultés en même temps que son profil.
- Le nombre d'emprunts actifs par utilisateur est faible et borné.
- Les mises à jour concernent principalement l'utilisateur (ajout ou retour d'un emprunt).
- L'emprunt n'a pas d'existence pertinente sans l'utilisateur associé.
- Embedding permet de réduire les requêtes et simplifie la lecture.

Listing 5 – Utilisateur avec des emprunts

```
{
  _id: 13,
  nom: 'LAKROUZ',
  prenom: 'SARAH',
  email: 'sarrouz@gmail.com',
  mot_de_passe: '123',
  role: 'admin',
  emprunts: [
    {
      livre_id: 1,
      date_emprunt: ISODate('2025-11-30T23:10:44.409Z'),
      date_retour: ISODate('2025-12-15T23:10:44.409Z'),
      statut: 'en cours'
    },
    {
      livre_id: 54,
      date_emprunt: ISODate('2025-11-30T23:14:04.288Z'),
      date_retour: ISODate('2025-12-15T23:14:04.288Z'),
      statut: 'en cours'
    }
  ]
}
```

4. Relation : Livre ↔ Emprunts

Type : Référence

Justification :

- Un livre peut avoir un très grand nombre d’emprunts au fil du temps (historique important).
- Embedding ferait grossir le document “livre” de manière excessive.
- Les emprunts peuvent être consultés séparément pour l’historique de la bibliothèque.
- Favorise une structure évolutive et performante.

Listing 6 – Emprunt avec Livre (référence)

```
{
  "_id": 1,
  "utilisateur_id": 1,
  "livre_id": 1,
  "date_emprunt": "2025-10-01T00:00:00.000Z",
  "date_retour": "2025-10-10T00:00:00.000Z",
  "statut": "rendu"
},
```

Listing 7 – Livre stocké séparément

```
{
  "_id": 1,
  "titre": "Introduction à MongoDB",
  "auteur_id": 1,
  "categorie_id": 1,
  "annee": 2023,
  "stock": 5,
  "disponible": true
},
```

5. Relation : Livre ↔ Formats (PDF, Audio, Papier)

Type : Embedded

Justification :

- Les formats sont de petits sous-documents liés directement au livre.
- Ils sont toujours consultés avec les informations du livre.
- Le nombre de formats est très faible (en général 2 à 4).
- Mise à jour simple : le format dépend entièrement du livre.

Listing 8 –Livre avec le format (embedding)

```
{
  "_id": 1,
  "titre": "Introduction à MongoDB",
  "auteur_id": 1,
  "categorie_id": 1,
  "annee": 2023,
  "stock": 5,
  "disponible": true,
  "formats": [
    {
      "type": "eLivre",
      "editeur": "Éditeur 1",
      "langue": "FR",
      "resume": "Résumé pour Introduction à MongoDB (version eLivre)",
      "date_publication": "2023-01-01",
      "nombre_pages": 101
    },
    {
      "type": "Livre audio",
      "editeur": "Éditeur 1",
      "langue": "FR",
      "resume": "Résumé pour Introduction à MongoDB (version audio)",
      "date_publication": "2023-02-01",
      "duree_minutes": 62
    },
    {
      "type": "Livre imprimé",
      "editeur": "Éditeur 1",
      "langue": "FR",
      "resume": "Résumé pour Introduction à MongoDB (version imprimé)",
      "date_publication": "2023-03-01",
      "nombre_pages": 101
    }
  ]
},
```

4 Explication des requêtes sans l'agrégation

4.1. Afficher et ajouter les livres dans la catalogue de la bibliothèque

La fonctionnalité principale consiste à gérer les livres dans la bibliothèque. Grâce à cette requête, on peut afficher les livres ou en ajouter de nouveaux.

```
bibliotheque> db.livres.find()
[
  {
    _id: 1,
    titre: 'Introduction à MongoDB',
    auteur_id: 1,
    categorie_id: 1,
    annee: 2023,
    stock: 1,
    disponible: true
  },
  ...
]
```

```
bibliotheque> db.livres.insertOne({
titre: "Le Petit Prince",
auteur: { nom: "Antoine de Saint-Exupéry" },
categorie: { nom: "Littérature" },
annee: 1943,
stock: 5,
disponible: true
})
```

```
@router.get("/")
def get_livres():
    return list(livres.find())

@router.post("/")
def add_livre(data: Livre):
    livre_dict = data.dict()

    # Si l'utilisateur ne fournit pas d'_id
    if not livre_dict.get("_id"):
        # Cherche le dernier _id
        dernier_livre = livres.find({"_id": {"$type": "int"}}).sort("_id", -1).limit(1)
        dernier_livre = list(dernier_livre)
        livre_dict["_id"] = (dernier_livre[0]["_id"] + 1) if dernier_livre else 1

    livres.insert_one(livre_dict)
    return {"message": "Livre ajouté", "livre": livre_dict}
```

4.2. Afficher et ajouter les utilisateurs

Également, dans la bibliothèque, on a des utilisateurs de deux types de rôles : étudiant et administrateur. Les étudiants peuvent seulement emprunter des livres, et les administrateurs peuvent gérer le catalogue des livres en ajoutant de nouveaux ouvrages.

```
bibliotheque> db.utilisateurs.find()
[
  {
    _id: 2,
    nom: 'Bernard',
    prenom: 'Paul',
    email: 'paul.bernard@example.com',
    mot_de_passe: 'password2',
    role: 'etudiant'
  }, ...
]
```

```
db.utilisateurs.insertOne({
  nom: "Durand",
  prenom: "Sophie",
  email: "sophie.durand2@example.com",
  mot_de_passe: "password123",
  role: "etudiant"
})
```

```
# Récupérer les utilisateurs
@router.get("/")
def get_utilisateurs():
    return list(
        utilisateurs.find(
            {},
            {"_id": 1, "nom": 1, "prenom": 1, "email": 1, "role": 1}
        )
    )
```

```
# Ajouter un nouvel utilisateur (inscription)
@router.post("/")
def add_utilisateur(data: Utilisateur):
    # Récupérer le dernier utilisateur
    dernier_utilisateur = utilisateurs.find_one(sort=[("_id", -1)])
    data_dict = data.dict()
    data_dict["_id"] = (dernier_utilisateur["_id"] + 1) if dernier_utilisateur else 1

    utilisateurs.insert_one(data_dict)
    return {"message": "Utilisateur ajouté", "id": data_dict["_id"]}
```

4.3. Développement des routes (emprunter et rendre un livre)

Dans notre système de gestion de bibliothèque, nous devons permettre aux utilisateurs d'emprunter un livre lorsqu'il est disponible, puis de le restituer afin de mettre à jour le stock et l'historique des emprunts. Ces routes assurent le suivi complet du cycle d'un livre, en vérifiant la disponibilité, en enregistrant l'emprunt, puis en rétablissant la disponibilité lors du retour.

```
db.emprunts.insertOne({
  utilisateur_id: 2,
  livre_id: 1,
  date_emprunt: new Date(),
  date_retour: new Date(new Date().getTime() + 15*24*60*60*1000),
  statut: "en cours"
})
```

```
db.emprunts.updateOne(
  { utilisateur_id: 2, livre_id: 1, statut: "en cours" },
  { $set: { statut: "rendu", date_retour: new Date() } }
)
```

```
@router.post("/emprunter")
def emprunter_livre(req: EmprunterRequest):
    user = get_user(req.utilisateur_id)
    livre = livres.find_one({"_id": req.livre_id})

    if not livre:
        raise HTTPException(404, "Livre introuvable")
    if livre.get("stock", 0) <= 0:
        raise HTTPException(400, "Livre non disponible")

    #Document pour l'utilisateur
    emprunt_user = EmpruntEmbedded(
        livre_id=req.livre_id,
        date_emprunt=datetime.now(),
        date_retour=datetime.now() + timedelta(days=15),
        statut="en cours"
    )

    emprunt_doc = {
        "utilisateur_id": req.utilisateur_id,
        "livre_id": req.livre_id,
        "date_emprunt": emprunt_user.date_emprunt,
        "date_retour": emprunt_user.date_retour,
        "statut": "en cours"
    }

    emprunts.insert_one(emprunt_doc)
```

```
def rendre_livre(req: RendreRequest):
    emprunt = user["emprunts"][req.index_emprunt]

    if emprunt["statut"] != "en cours":
        raise HTTPException(400, "Cet emprunt n'est pas en cours")

    date_emprunt = datetime.fromisoformat(emprunt["date_emprunt"]) \
        if isinstance(emprunt["date_emprunt"], str) else emprunt["date_emprunt"]

    retard = (datetime.now() - date_emprunt).days > MAX_DUREE_JOURS

    utilisateurs.update_one(
        {"_id": req.utilisateur_id},
        {
            "$set": {
                f"emprunts.{req.index_emprunt}.statut": "retard" if retard else "rendu",
                f"emprunts.{req.index_emprunt}.date_retour": datetime.now()
            }
        }
    )

    emprunts.update_one(
        {"utilisateur_id": req.utilisateur_id, "livre_id": emprunt["livre_id"], "statut": "en cours"},
        {"$set": {
            "statut": "retard" if retard else "rendu",
            "date_retour": datetime.now()
        }}
    )
)
```

4.4. Mise à jour automatique des emprunts (disponibilités, dates, retards)

Le système met à jour automatiquement l'état des emprunts en temps réel : il ajuste la disponibilité des livres dans le catalogue, actualise les dates de retour, et détecte les retards afin de signaler les emprunts dépassant la durée maximale autorisée. Cela permet de maintenir la cohérence des informations et de faciliter la gestion de la bibliothèque.

```
db.livres.find(
    { disponible: true },
    { _id: 0, titre: 1, "auteur.nom": 1, "categorie.nom": 1, annee: 1, stock: 1 }
)

[
    { titre: 'Introduction à MongoDB', annee: 2023, stock: 1 },
    { titre: 'Bases de Données SQL', annee: 2019, stock: 1 },
    { titre: 'Les Misérables', annee: 1862, stock: 2 }
]
```

```
def rendre_livre(req: RendreRequest):
    emprunt = user["emprunts"][req.index_emprunt]

    if emprunt["statut"] != "en cours":
        raise HTTPException(400, "Cet emprunt n'est pas en cours")

    date_emprunt = datetime.fromisoformat(emprunt["date_emprunt"]) \
        if isinstance(emprunt["date_emprunt"], str) else emprunt["date_emprunt"]

    retard = (datetime.now() - date_emprunt).days > MAX_DUREE_JOURS

    utilisateurs.update_one(
        {"_id": req.utilisateur_id},
        {
            "$set": {
                f"emprunts.{req.index_emprunt}.statut": "retard" if retard else "rendu",
                f"emprunts.{req.index_emprunt}.date_retour": datetime.now()
            }
        }
    )

    emprunts.update_one(
        {"utilisateur_id": req.utilisateur_id, "livre_id": emprunt["livre_id"], "statut": "en cours"},
        {"$set": {
            "statut": "retard" if retard else "rendu",
            "date_retour": datetime.now()
        }}
    )
)
```


4.5. Gestion des retards et des emprunts

Le système identifie automatiquement les emprunts en retard (au bout de 15 jours) et met à jour leur statut dans la base de données. Les utilisateurs peuvent consulter leurs emprunts en cours et leur état, tandis que les administrateurs ont la possibilité de suivre l'ensemble des emprunts et des retards, facilitant ainsi la gestion efficace des livres et le rappel des ouvrages non rendus à temps.

```
db.emprunts.find(
  { statut: "retard" },
  { _id: 0, utilisateur_id: 1, date_emprunt: 1, date_retour: 1 }
)

[
  {
    utilisateur_id: 4,
    date_emprunt: '2025-10-15T00:00:00.000Z',
    date_retour: null
  }
]
```

```
9 def rendre_livre(req: RendreRequest):
10     "statut": "retard" if retard else "normal",
11     "date_retour": datetime.now()
12     }
13 )
14
15 #Mettre à jour stock
16 livres.update_one({"_id": emprunt["livre_id"]}, {"$inc": {"stock": +1}})
17 livres.update_one({"_id": emprunt["livre_id"]}, {"$set": {"disponible": True}})
18
19 return {"message": "Livre rendu", "retard": retard}
20
21 # -----|
22 # ROUTE : liste par utilisateur
23 # -----|
24 @router.get("/user/{user_id}")
25 def emprunts_par_utilisateur(user_id: int):
26     user = get_user(user_id)
27     emprunts_liste = []
28
29     for emp in user.get("emprunts", []):
30         livre = livres.find_one({"_id": emp["livre_id"]})
31         emprunts_liste.append({
32             "livre_id": emp["livre_id"],
33             "titre": livre["titre"] if livre else "Titre inconnu",
34             "date_emprunt": emp["date_emprunt"],
35             "date_retour": emp.get("date_retour"),
36             "statut": emp["statut"]
37         })
```

5 Explication des requêtes avec l'agrégation

5.1. Top des livres les plus empruntés

1. Regroupement des emprunts par livre

Chaque emprunt est regroupé selon l'identifiant du livre correspondant.
Cela permet de compter combien de fois chaque livre a été emprunté.

2. Calcul du nombre total d'emprunts par livre

Pour chaque groupe, on additionne le nombre d'occurrences, ce qui donne le total des emprunts pour chaque livre.

3. Tri du classement

Les livres sont ensuite triés du plus emprunté au moins emprunté.

4. Limitation du nombre de résultats

Le système retourne uniquement les premiers résultats du classement (par exemple les 5 meilleurs).

5. Récupération des informations du livre

Une jointure est réalisée entre la collection des emprunts et la collection des livres.
Cette étape permet d'ajouter les informations détaillées du livre (titre, auteur, etc.) au résultat final.

6. Nettoyage et formatage des données

Seules les informations suivantes sont conservées afin de fournir un résultat clair :

- le titre du livre,
- son auteur,
- et le nombre total d'emprunts.

```
db.emprunts.aggregate([
  {"$group": {"_id": "$livre_id", "nb_emprunts": {"$sum": 1}}},
  {"$sort": {"nb_emprunts": -1}},
  {"$limit": 5},
  {"$lookup": {"from": "livres", "localField": "_id", "foreignField": "_id", "as": "livre"}},
  {"$unwind": "$livre"},
  {"$project": {"livre._id": 1, "livre.titre": 1, "livre.auteur": 1, "nb_emprunts": 1}}
])
```

```

# 1. Top des livres les plus empruntés
@router.get("/top-livres")
def top_livres(limit: int = 5):
    try:
        pipeline = [
            {"$group": {
                "_id": "$livre_id",
                "nb_emprunts": {"$sum": 1}
            }},
            {"$sort": {"nb_emprunts": -1}},
            {"$limit": limit},
            {"$lookup": {
                "from": "livres",
                "localField": "_id",
                "foreignField": "_id",
                "as": "livre"
            }},
            {"$unwind": "$livre"},
            {"$project": {
                "livre._id": 1,
                "livre.titre": 1,
                "livre.auteur": 1,
                "nb_emprunts": 1
            }}
        ]
        return list(emprunts.aggregate(pipeline))
    except:
        pass

```

5.2. Nombre d'emprunts par catégorie

1. Association entre emprunts et livres

Chaque emprunt est enrichi avec les informations du livre correspondant grâce à une jointure avec la collection des livres.

Cette étape permet notamment de récupérer la catégorie du livre emprunté.

2. Regroupement des emprunts par catégorie

Une fois les livres liés à chaque emprunt, tous les emprunts sont regroupés selon l'identifiant de la catégorie.

Cela permet de compter combien d'emprunts appartiennent à chaque catégorie.

3. Calcul du total d'emprunts par catégorie

Pour chaque catégorie, on additionne les emprunts afin d'obtenir un total du nombre de prêts réalisés pour cette catégorie.

4. Association avec la collection des catégories

Pour rendre le résultat plus lisible, une deuxième jointure est effectuée avec la collection des catégories.

Cette étape permet de récupérer le nom complet de la catégorie et pas seulement son identifiant.

5. Mise en forme du résultat final

Le résultat affiché contiendra les informations suivantes:

- l'identifiant de la catégorie,
- son nom,
- le nombre total d'emprunts associés.

```

db.emprunts.aggregate([
{"$lookup": {"from": "livres", "localField": "livre_id", "foreignField": "_id", "as":
"livre"}},

{"$unwind": "$livre"},

{"$group": {"_id": "$livre.categorie_id", "nb_emprunts": {"$sum": 1}}},

{"$lookup": {"from": "categories", "localField": "_id", "foreignField": "_id", "as":
"categorie"}},

{"$unwind": "$categorie"},

{"$project": {"_id": 0, "categorie_id": "$_id", "categorie_nom":
"$categorie.nom", "nb_emprunts": 1}}])

```

```

def emprunts_par_categorie():
    try:
        pipeline = [
            # Lier les emprunts avec les livres
            {"$lookup": {
                "from": "livres",
                "localField": "livre_id",
                "foreignField": "_id",
                "as": "livre"
            }},
            {"$unwind": "$livre"},

            # Grouper par catégorie
            {"$group": {
                "_id": "$livre.categorie_id",
                "nb_emprunts": {"$sum": 1}
            }},

            # Lier avec la collection categories pour obtenir le nom
            {"$lookup": {
                "from": "categories",
                "localField": "_id",
                "foreignField": "_id",
                "as": "categorie"
            }},
            {"$unwind": "$categorie"},

            # Projeter le résultat final
            {"$project": {
                "_id": 0,
                "categorie_id": "$_id",
                "categorie_nom": "$categorie.nom",
                "nb_emprunts": 1
            }}
        ]
    except:
        return list(emprunts.aggregate(pipeline))

```

5.3. Durée moyenne d'emprunt par utilisateur

1. Filtrer les emprunts rendus

Le traitement commence par sélectionner uniquement les emprunts pour lesquels une date de retour est renseignée.

2. Calculer la durée de chaque emprunt

Pour chaque emprunt considéré, la durée est calculée en jours à partir de la différence entre la date d'emprunt et la date de retour.

Cette durée est ajoutée comme nouvelle information dans les documents traités.

3. Regrouper les emprunts par utilisateur

Tous les emprunts d'un même utilisateur sont regroupés ensemble afin de pouvoir effectuer des statistiques individuelles.

4. Calculer la durée moyenne d'emprunt par utilisateur

On calcule par la suite :

- la durée moyenne des emprunts de l'utilisateur,
- ainsi que le nombre total d'emprunts utilisés dans le calcul.

```
db.emprunts.aggregate([
{"$match": {"date_retour": {"$ne": None}}},

{"$project": {"utilisateur_id": 1,"duree_jours": {"$dateDiff": {"startDate":
{"$toDate": "$date_emprunt"},"endDate": {"$toDate": "$date_retour"},"unit":
"day"}}}},

{"$group": {"_id": "$utilisateur_id","moyenne_duree_jours": {"$avg":
"$duree_jours"},"nb_emprunts": {"$sum": 1}}}]])
```

```
def duree_moyenne_utilisateur():
    try:
        pipeline = [
            {"$match": {"date_retour": {"$ne": None}}},
            {"$project": {
                "utilisateur_id": 1,
                "duree_jours": {
                    "$dateDiff": {
                        "startDate": {"$toDate": "$date_emprunt"},
                        "endDate": {"$toDate": "$date_retour"},
                        "unit": "day"
                    }
                }
            }},
            {"$group": {
                "_id": "$utilisateur_id",
                "moyenne_duree_jours": {"$avg": "$duree_jours"},
                "nb_emprunts": {"$sum": 1}
            }}
        ]
    return list(emprunts.aggregate(pipeline))
```

5.4. Durée moyenne d'emprunt par livre

1. Sélection des emprunts rendus

Seuls les emprunts ayant une date de retour sont pris en compte.

2. Calcul de la durée de chaque emprunt

Pour chaque emprunt sélectionné, le nombre de jours entre la date d'emprunt et la date de retour est calculé.

Cette valeur est ajoutée au document sous la forme d'un champ « durée jours ».

3. Regroupement par livre

Tous les emprunts liés à un même livre sont regroupés afin de calculer des statistiques sur ce titre.

4. Calcul des statistiques par livre

Pour chaque livre, deux informations principales sont produites :

- la durée moyenne d'emprunt en jours,
- le nombre total d'emprunts utilisés dans le calcul.

5. Tri des livres

Les résultats sont triés par nombre d'emprunts, du plus emprunté au moins emprunté.

Cela permet d'observer en priorité les livres les plus empruntés.

```
db.emprunts.aggregate([
{"$match": {"date_retour": {"$ne": None}}},

{"$project": {"livre_id": 1,"duree_jours": {"$dateDiff": {"startDate": {"$toDate":
"$date_emprunt"},"endDate": {"$toDate": "$date_retour"},"unit": "day"}}}},

{"$group": {"_id": "$livre_id","moyenne_duree_jours": {"$avg":
"$duree_jours"},"nb_emprunts": {"$sum": 1}}},

{"$sort": {"nb_emprunts": -1}}])
```

```
def duree_moyenne_livre():
    try:
        pipeline = [
            {"$match": {"date_retour": {"$ne": None}}},
            {"$project": {
                "livre_id": 1,
                "duree_jours": {
                    "$dateDiff": {
                        "startDate": {"$toDate": "$date_emprunt"},
                        "endDate": {"$toDate": "$date_retour"},
                        "unit": "day"
                    }
                }
            }},
            {"$group": {
                "_id": "$livre_id",
                "moyenne_duree_jours": {"$avg": "$duree_jours"},
                "nb_emprunts": {"$sum": 1}
            }},
            {"$sort": {"nb_emprunts": -1}}
        ]
        return list(emprunts.aggregate(pipeline))
```

5.5 Top des écrivains les plus empruntés

1. Association des emprunts avec les livres

Chaque emprunt est enrichi avec les informations du livre correspondant grâce à une jointure avec la collection des livres.

Cela permet de connaître quel livre a été emprunté à chaque fois.

2. Association des livres avec leurs auteurs

Une seconde jointure est réalisée pour relier chaque livre à son auteur.

Cela permet d'attribuer chaque emprunt au bon auteur.

3. Regroupement par auteur

Les emprunts sont regroupés par auteur afin de compter combien d'emprunts concernent chacun d'eux.

4. Calcul du nombre total d'emprunts par auteur

Pour chaque auteur, on calcule le nombre total d'emprunts de ses livres.

5. Tri des auteurs les plus empruntés

Les résultats sont triés par ordre décroissant du nombre d'emprunts, ce qui permet d'afficher immédiatement les auteurs les plus consultés.

6. Limitation des résultats

Le pipeline peut limiter le nombre d'auteurs affichés, afin de n'afficher par exemple que les 5 ou 10 auteurs les plus empruntés.

```
db.emprunts.aggregate([
  {"$lookup": {"from": "livres", "localField": "livre_id", "foreignField": "_id", "as": "livre"}}, {"$unwind": "$livre"},

  {"$lookup": {"from": "auteurs", "localField": "livre.auteur_id", "foreignField": "_id", "as": "auteur"}},

  {"$unwind": "$auteur"},

  {"$group": {"_id": "$auteur.nom", "nb_emprunts": {"$sum": 1}}},

  {"$sort": {"nb_emprunts": -1}},
  {"$limit": 5}
])
```

```

def top_auteurs(limit: int = 5):
    try:
        pipeline = [
            # Lier chaque emprunt au livre correspondant
            {"$lookup": {
                "from": "livres",
                "localField": "livre_id",
                "foreignField": "_id",
                "as": "livre"
            }},
            {"$unwind": "$livre"},

            # Lier chaque livre à son auteur
            {"$lookup": {
                "from": "auteurs",
                "localField": "livre.auteur_id",
                "foreignField": "_id",
                "as": "auteur"
            }},
            {"$unwind": "$auteur"},

            # Grouper par auteur et compter le nombre d'emprunts
            {"$group": {
                "_id": "$auteur.nom",
                "nb_emprunts": {"$sum": 1}
            }},

            # Trier par nombre d'emprunts décroissant
            {"$sort": {"nb_emprunts": -1}},

            # Limiter le nombre de résultats
            {"$limit": limit}
        ]
    return list(emprunts.aggregate(pipeline))

```


6 Tableau des charges de travail

Dans l'équipe, nous avons réparti le travail du projet entre les quatre membres. Chacun a choisi les tâches selon ses préférences et ses compétences. Cependant, pour les décisions les plus cruciales, nous nous sommes toujours réunis afin de décider ensemble, notamment pour la définition du schéma de la base de données et les choix entre les relations de type embedded ou par référence.

Tâche / Fonctionnalité	Personne responsable
Mise en place du projet backend (FastAPI)	Yelizaveta
Connexion à Mongoddb	Yelizaveta
Création des collections	Yelizaveta
Création des modèles Pydantic	Yelizaveta
Définition du schéma Mongoddb (choix entre références et documents imbriqués)	Yelizaveta, Sarah, Ilayda, Amina
CRUD Livres	Yelizaveta
CRUD utilisateurs	Yelizaveta
Développement des routes (emprunter et rendre un livre)	Sarah
Mise à jour automatique (disponibilités, dates, retards)	Sarah
Gestion des retards et des emprunts	Sarah
Recherche avancée	Sarah
Top des livres le plus empruntés	Ilayda
Nombre d'emprunts par catégorie	Ilayda
Durée moyenne d'emprunt par utilisateur	Ilayda
Durée moyenne d'emprunt par livre	Ilayda
Top des écrivains les plus empruntés	Ilayda
Connexion utilisateur	Amina
Frontend React	Amina

7 Travail avec Postman

Également, pendant le développement du back-end, nous avons créé une interface Postman où nous avons testé les requêtes et les appels à l'API, avec la possibilité de sauvegarder les requêtes fréquentes. Dans notre dépôt GitHub, nous avons ajouté un fichier que l'on peut importer dans Postman pour consulter et tester ces requêtes.

The screenshot displays the Postman interface with the following details:

- Collection:** Bibliothèque_API / Statistiques / Top auteurs empruntés
- Method:** GET
- URL:** {{base_url}}/stats/top-auteurs
- Params:** A table with columns Key, Value, and Description. The first row contains 'Key' and 'Value'.
- Body:** A JSON array of objects representing author statistics. The response is:

```
[{"_id": "Marie Curie", "nb_emprunts": 2}, {"_id": "Albert Einstein", "nb_emprunts": 1}, {"_id": "Jean Dupont", "nb_emprunts": 1}]
```
- Status:** 200 OK

8 Conclusion

Ce projet nous a permis de concevoir et de développer une application complète de gestion d'une bibliothèque en ligne basée sur une base de données NoSQL orientée documents avec MongoDB. L'identification des entités principales (livres, auteurs, catégories, utilisateurs et emprunts) ainsi que l'analyse de leurs relations nous ont permis de construire un modèle de données cohérent, performant et adapté aux besoins réels de l'application.

Les choix effectués entre documents imbriqués (embedded) et références ont été justifiés en fonction de critères précis tels que la fréquence des requêtes, la cardinalité, la performance et la maintenabilité. Les différentes fonctionnalités mises en place, comme la gestion des livres, des utilisateurs, des emprunts et des retards, ainsi que les requêtes d'agrégation (top des livres, statistiques par catégorie, durée moyenne d'emprunt, auteurs les plus empruntés), montrent l'efficacité et la flexibilité de MongoDB pour ce type d'application.

La répartition des tâches au sein du groupe a permis une bonne organisation du travail et une avancée efficace du projet, aussi bien sur le plan technique que fonctionnel.

En perspective, plusieurs améliorations peuvent être envisagées, notamment une meilleure gestion des formats des livres (ebook, audio, papier) avec des métadonnées plus détaillées, l'ajout de plusieurs rôles dans la base de données (bibliothécaire, administrateur, modérateur, etc.) afin d'affiner les droits d'accès, ainsi que la possibilité de prolonger un emprunt directement depuis l'application. D'autres évolutions pourraient également inclure l'envoi de notifications automatiques pour les retards, un système de recommandations ou encore un tableau de bord statistique avancé.

Ainsi, ce projet constitue une base solide pour une application de bibliothèque moderne, évolutive et bien adaptée aux besoins des utilisateurs.